

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Deepan Kumar P	Reg. No.:	192312498
Section / Batch:	BE ECE 4 th year	Date:	25/08/2026

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system receives interrupt requests from sensors, communication modules, and storage devices at unpredictable intervals. Develop pseudocode for a nested interrupt handling mechanism that allows a higher-priority interrupt to pre-empt a lower-priority service routine. The algorithm should save the current processor context, service the higher-priority interrupt, restore the interrupted routine, and ensure that pending interrupts are handled correctly.
2. A data-intensive embedded system must transfer a large amount of information between memory and multiple I/O devices without continuously occupying the processor. Write pseudocode for a scatter-gather DMA mechanism that transfers data stored in multiple non-contiguous memory locations. The algorithm should maintain a list of memory descriptors, initiate successive transfers, update descriptor status, and handle completion and error conditions.
3. A computer system supports several I/O devices that generate data at different rates, causing temporary variations in the amount of data waiting for processing. Design pseudocode for an I/O buffering mechanism that manages incoming and outgoing data using appropriate buffer allocation and release strategies. The algorithm should monitor buffer occupancy, handle buffer-full and buffer-empty conditions, and prevent data loss while maintaining efficient device communication.
4. A high-speed embedded platform contains multiple communication devices competing for shared system resources. Develop pseudocode for a priority-based resource allocation mechanism that assigns communication resources according to device urgency and bandwidth requirements. The algorithm should dynamically update priorities, allocate resources to eligible devices, release resources after transmission, and prevent low-priority devices from being indefinitely delayed.
5. A real-time system must transfer data from several peripheral devices while meeting strict timing constraints. Write pseudocode for a real-time I/O scheduling mechanism that determines when each device should perform data transfers based on deadlines, transfer duration, priority, and available system bandwidth. The algorithm should schedule time-critical transfers first, monitor missed deadlines, and reschedule pending requests to improve overall system responsiveness. **Code of Conduct**

I Deepan Kumar P(192312498) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge

1.Nested Interrupt Handling Mechanism

Pseudocode

```
BEGIN
Initialize interrupt controller
Initialize interrupt priority levels
Enable interrupts
WHILE system is running
    IF interrupt request is received THEN
        Identify interrupt source
        Determine interrupt priority
        IF interrupt priority is higher than current priority THEN
            Save current processor context
            Save current interrupt priority
            Disable lower-priority interrupts
            Set current priority to received priority
            Execute interrupt service routine
            Check for pending higher-priority interrupts
            WHILE pending higher-priority interrupt exists
                Select highest-priority pending interrupt
                Save current processor context
                Service selected interrupt
                Restore previous processor context
            END WHILE
            Restore previous interrupt priority
            Restore interrupted processor context
            Resume interrupted routine
        ELSE
            Store interrupt request in pending interrupt queue
        END IF
    END IF
END WHILE
END
```

2. Scatter-Gather DMA Mechanism

Pseudocode

```
BEGIN
Create descriptor list
FOR each memory block
    Store source address
    Store destination address
    Store transfer size
    Set descriptor status to READY
END FOR
Set current descriptor to first descriptor
WHILE descriptors remain
    IF current descriptor status is READY THEN
        Load source address
        Load destination address
        Load transfer size
```

```

Start DMA transfer
IF DMA transfer is successful THEN
    Set descriptor status to COMPLETED
    Move to next descriptor
ELSE
    Set descriptor status to ERROR
    Record error information
    IF error can be recovered THEN
        Retry DMA transfer
    ELSE
        Stop DMA operation
        Report error
        EXIT WHILE
    END IF
END IF
END IF
ELSE
    Move to next descriptor
END IF
END WHILE
IF all descriptors are COMPLETED THEN
    Generate DMA completion interrupt
    Notify processor
END IF
END

```

3. I/O Buffering Mechanism

Pseudocode

```

BEGIN
Initialize input buffer
Initialize output buffer
Set buffer capacity
Set input and output pointers
WHILE system is running
    IF input device has data THEN
        IF input buffer is not full THEN
            Store incoming data in input buffer
            Update input pointer
            Increase buffer occupancy
        ELSE
            Report buffer full condition
            IF additional storage is available THEN
                Move data to additional buffer
            ELSE
                Apply flow control
                Temporarily stop incoming data
            END IF
        END IF
    END IF
END IF
IF output device is ready THEN
    IF output buffer is not empty THEN

```

```

    Read data from output buffer
    Send data to output device
    Update output pointer
    Decrease buffer occupancy
ELSE
    Report buffer empty condition
    Keep output device waiting
END IF
END IF
IF buffer occupancy is very high THEN
    Enable flow control
END IF
IF buffer occupancy is low THEN
    Disable flow control
END IF
END WHILE
END

```

4. Priority-Based Resource Allocation Mechanism

Pseudocode

```

BEGIN
Initialize device request queue
Initialize device priorities
Initialize available bandwidth
WHILE communication system is active
    Receive resource requests
    FOR each waiting device
        Increase waiting time
        IF waiting time exceeds threshold THEN
            Increase device priority
        END IF
        IF device has an urgent request THEN
            Increase device priority
        END IF
    END FOR
    Sort waiting devices according to updated priority
    FOR each device in priority order
        IF required bandwidth is available THEN
            Allocate communication resource
            Start transmission
            WHILE transmission is not complete
                Continue data transfer
            END WHILE
            Release communication resource
            Update device status
            Reset waiting time
        ELSE
            Keep device in waiting queue
        END IF
    END FOR
END WHILE

```

END

5.Real-Time I/O Scheduling Mechanism

Pseudocode

BEGIN

Initialize I/O request queue

Initialize current system time

Initialize available bandwidth

WHILE system is running

 Receive new I/O requests

 FOR each new request

 Store device information

 Store deadline

 Store transfer duration

 Store priority

 Store bandwidth requirement

 END FOR

 FOR each pending request

 IF current time is greater than deadline THEN

 Mark request as MISSED

 Increase request priority

 Add request to urgent queue

 END IF

 END FOR

 Sort pending requests according to:

 Earliest deadline

 Highest priority

 Shortest transfer duration

 Required bandwidth

 FOR each request in sorted order

 IF required bandwidth is available AND

 current time plus transfer duration is within deadline THEN

 Allocate bandwidth

 Start data transfer

 Update current time

 Reduce available bandwidth

 IF transfer is completed THEN

 Mark request as COMPLETED

 Release allocated bandwidth

 END IF

 ELSE

 Keep request in pending queue

 END IF

 END FOR

 IF pending request has a critical deadline THEN

 Give request higher priority

 Reschedule remaining transfers

 END IF

 Restore available bandwidth

END WHILE

END